

Lineamiento de Nomenclatura de Servicios AWS

1. Objetivo

Este documento establece la nomenclatura estándar y las mejores prácticas para la creación y administración de servicios en AWS. El objetivo es garantizar organización, trazabilidad, seguridad y escalabilidad en la infraestructura cloud.

2. Nomenclatura Estándar

La nomenclatura se basa en un formato estructurado que permite identificar de manera clara cada recurso.

2.1. Estructura General

La siguiente nomenclatura usa guiones(-) para denotar jerarquía y para nombres compuesto es obligatorio el uso de guión bajo(_)

```
1 <tipo-recurso>-<siglaververticalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Elemento	Definición	Ejemplo
tipo-recurso	Tipo de servicio AWS (vpc , subnet , api , lambda , etc.).	vpc
siglaververticalnegocio	Identifica la vertical de negocio (col = Casino Online, caf = Casino Físico, adm =administración, acity =transversal).	col
nombre-proyecto identificado-recurso	Nombre del proyecto o un identificador único del recurso.	calimaco
nombre-personalizado	Identificador único del recurso.	app-backend

✓ Ejemplo completo:

- vpc-col-calimaco-default
- vpc-col-calimaco-sandbox
- gateway-caf-dm_alertas-api_bot
- lambda-col-calimaco-api_validatesesion_calimaco

3. Lineamientos para Servicios AWS

3.1 Servicios IAM

1. Políticas IAM (policyiam)

Las políticas IAM definen permisos para usuarios, grupos o roles.

Formato:

```
1 policyiam-<siglaververticalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

- policyiam-col-calimaco-admin-fullaccess → Política de acceso total para administradores de "calimaco".
- policyiam-caf-dm_alerta-readonly → Política de solo lectura para el servicio "dm_alerta".
- policyiam-adm-backend-deployment → Política para despliegues en backend de administración.

2. Roles IAM (roleiam)

Los roles IAM se utilizan para otorgar permisos a entidades confiables (usuarios, servicios de AWS, etc.).

Formato:

```
1 roleiam-<siglaververticalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

- `roleiam-col-calimaco-ecs-task-execution` → Rol para ejecución de tareas en ECS.
- `roleiam-adm-backend-lambda-execution` → Rol para ejecución de funciones Lambda en backend.
- `roleiam-caf-dm_alerta-s3-access` → Rol que permite acceso a S3 para el servicio "dm_alerta".

3. Grupos IAM (`groupiam`)

Los grupos IAM agrupan usuarios con permisos similares.

Formato:

```
1 groupiam-<siglaverticalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

- `groupiam-col-calimaco-admins` → Grupo de administradores del proyecto "calimaco".
- `groupiam-adm-backend-devs` → Grupo de desarrolladores con acceso al backend.
- `groupiam-caf-dm_alerta-analysts` → Grupo de analistas con acceso a "dm_alerta".

4. Usuarios IAM (`useriam`)

Los usuarios IAM representan identidades específicas que requieren credenciales para autenticarse en AWS.

Formato:

```
1 useriam-<siglaverticalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

- `useriam-col-calimaco-jdoe` → Usuario "jdoe" con acceso al proyecto "calimaco".
- `useriam-adm-backend-serviceaccount` → Cuenta de servicio para backend de administración.
- `useriam-caf-dm_alerta-opsuser` → Usuario de operaciones para el servicio "dm_alerta".

5. Perfiles de Instancia IAM (`instprofiam`)

Los perfiles de instancia se asocian a instancias EC2 para otorgarles permisos mediante roles.

Formato:

```
1 instprofiam-<siglaverticalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

- `instprofiam-col-calimaco-appserver` → Perfil de instancia para servidores de aplicación de "calimaco".
- `instprofiam-adm-backend-ec2` → Perfil de instancia para servidores backend en EC2.
- `instprofiam-caf-dm_alerta-processing` → Perfil de instancia para procesamiento en "dm_alerta".

3.2 Servicios VPC

1. Redes Virtuales (VPC)

Las VPCs (Virtual Private Cloud) permiten la creación de redes privadas dentro de AWS.

Formato:

```
1 vpc-<siglaverticalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

```
1 vpc-col-calimaco-default      → VPC por defecto del proyecto "calimaco" en Casino Online.
2 vpc-caf-dm_alerta-prod       → VPC de producción para "dm_alerta" en Casino Físico.
3 vpc-adm-backend-sandbox      → VPC de pruebas para el backend de administración.
```

2. Subredes (Subnets)

Las subnets permiten segmentar redes dentro de una VPC.

Formato:

```
1 subnet-<siglaverticalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

```
1 subnet-col-calimaco-public-a  → Subnet pública en zona A para "calimaco".
2 subnet-col-calimaco-private-b → Subnet privada en zona B para "calimaco".
3 subnet-adm-backend-public-c   → Subnet pública en zona C para backend de administración.
```

3. Gateway de Internet (IGW)

El Internet Gateway permite a las instancias de una VPC acceder a internet.

Formato:

```
1 igw-<siglavericalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

```
1 igw-col-calimaco-main      → Internet Gateway principal para "calimaco".
2 igw-caf-dm_alerta-prod    → Internet Gateway en ambiente de producción para "dm_alerta".
```

4. NAT Gateway (NAT)

El NAT Gateway permite a las instancias privadas acceder a Internet sin ser accesibles desde el exterior.

Formato:

```
1 nat-<siglavericalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

```
1 nat-col-calimaco-public-a  → NAT Gateway en la zona pública A para "calimaco".
2 nat-adm-backend-public-b   → NAT Gateway en la zona pública B para backend de administración.
```

5. Tablas de Rutas (Route Tables - RT)

Las tablas de rutas definen las reglas de enrutamiento dentro de una VPC.

Formato:

```
1 rt-<siglavericalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

```
1 rt-col-calimaco-public     → Tabla de rutas para las subnets públicas de "calimaco".
2 rt-col-calimaco-private    → Tabla de rutas para las subnets privadas de "calimaco".
3 rt-adm-backend-default     → Tabla de rutas por defecto para backend de administración.
```

6. Grupos de Seguridad (Security Groups - SG)

Los Security Groups controlan el tráfico entrante y saliente de las instancias.

Formato:

```
1 sg-<siglavericalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

```
1 sg-col-calimaco-webserver  → Grupo de seguridad para servidores web en "calimaco".
2 sg-col-calimaco-database    → Grupo de seguridad para bases de datos en "calimaco".
3 sg-adm-backend-lambda      → Grupo de seguridad para funciones Lambda del backend de administración.
```

7. Listas de Control de Acceso de Red (NACLs)

Las NACLs (Network ACLs) controlan el tráfico a nivel de subnet.

Formato:

```
1 nacl-<siglavericalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

```
1 nacl-col-calimaco-public    → ACL para subnets públicas de "calimaco".
2 nacl-col-calimaco-private    → ACL para subnets privadas de "calimaco".
3 nacl-adm-backend-default    → ACL por defecto para backend de administración.
```

8. Elastic IPs (EIP)

Las direcciones IP elásticas permiten la asignación de IPs fijas a recursos en AWS.

Formato:

```
1 eip-<siglavericalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

```
1 eip-col-calimaco-nat-a      → Elastic IP asignada a NAT Gateway en zona A para "calimaco".
2 eip-adm-backend-bastion     → Elastic IP para el servidor bastión del backend de administración.
```

9. Interfaces de Red Elástica (ENI)

Las ENI permiten asignar múltiples interfaces de red a instancias EC2.

Formato:

```
1 eni-<siglavericalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

```
1 eni-col-calimaco-appserver    → ENI para servidores de aplicación de "calimaco".
2 eni-adm-backend-database     → ENI para base de datos del backend de administración.
```

10. Peering de VPC (VPC Peering)

El VPC Peering conecta dos redes VPC para permitir comunicación entre ellas.

Formato:

```
1 peer-<siglavericalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

```
1 peer-acity-calimaco_to_bally-main    → Peering entre vpc calimaco y bally.
2 peer-acity-calimaco_to_administracion-prod → Peering entre vpc calimaco y administracion.
```

3.3 Servicios Lambda

1. Funciones Lambda

Las funciones Lambda permiten ejecutar código sin aprovisionar ni administrar servidores.

Formato:

```
1 lambda-<siglavericalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

```
1 lambda-col-calimaco-process_order    → Función Lambda para procesar órdenes en "calimaco".
2 lambda-caf-dm_alerta-send_notification → Lambda para enviar notificaciones en "dm_alerta".
3 lambda-adm-backend-data_ingestion    → Lambda para la ingesta de datos en el backend de administración.
```

2. Alias de Lambda

Los alias permiten gestionar versiones de Lambda con nombres legibles.

Formato:

```
1 lambdaalias-<siglavericalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>-<version>
```

Ejemplos:

```
1 lambdaalias-col-calimaco-process_order-prod → Alias "prod" para la función "process_order" en "calimaco".
2 lambdaalias-caf-dm_alerta-send_notification-dev → Alias "dev" para la función "send_notification" en "dm_alerta".
3 lambdaalias-adm-backend-data_ingestion-v2 → Alias "v2" para la función "data_ingestion".
```

3. Capas de Lambda (Layers)

Las **capas Lambda** permiten compartir código entre funciones.

Formato:

```
1 lambdalayer-<siglavericalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

```
1 lambdalayer-col-calimaco-utils    → Capa con utilidades comunes para "calimaco".
2 lambdalayer-caf-dm_alerta-logger  → Capa con funciones de logging para "dm_alerta".
3 lambdalayer-adm-backend-db_connector → Capa con el conector de base de datos para el backend de administración.
```

4. Permisos Lambda (Lambda Permissions)

Los permisos de Lambda permiten a otros servicios invocar una función Lambda.

Formato:

```
1 lambdaperm-<siglavericalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

```
1 lambdaperm-col-calimaco-api_gateway-invoke → Permiso para que API Gateway invoque la función Lambda.
2 lambdaperm-caf-dm_alerta-s3-event_trigger → Permiso para que S3 dispare la función Lambda.
3 lambdaperm-adm-backend-cloudwatch-log_access → Permiso para que CloudWatch acceda a la función Lambda.
```

5. Destinos Lambda (Lambda Destinations)

Los destinos Lambda configuran a dónde se envían los resultados de una ejecución.

Formato:

```
1 lambdarest-<siglavericalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

```
1 lambdaDest-col-calimaco-process_order-sqs → Destino SQS para resultados de "process_order".
2 lambdaDest-caf-dm_alerta-error_handling-sns → Destino SNS para manejar errores en "dm_alerta".
3 lambdaDest-adm-backend-eventbridge-log → Destino EventBridge para eventos de "data_ingestion".
```

6. Eventos de Disparadores Lambda (Lambda Triggers)

Los disparadores definen eventos que inician una ejecución de Lambda.

Formato:

```
1 lambdaTrigger-<siglaVerticalNegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

```
1 lambdaTrigger-col-calimaco-s3-new_order → Disparador S3 para nuevos pedidos en "calimaco".
2 lambdaTrigger-caf-dm_alerta-kinesis-stream → Disparador Kinesis para procesar logs en "dm_alerta".
3 lambdaTrigger-adm-backend-sqs-new_message → Disparador SQS para procesamiento de mensajes.
```

7. Configuración de Concurrencia Lambda

La configuración de concurrencia define el límite de ejecuciones simultáneas.

Formato:

```
1 lambdaConcur-<siglaVerticalNegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

```
1 lambdaConcur-col-calimaco-process_order-50 → Límite de 50 ejecuciones concurrentes en "process_order".
2 lambdaConcur-caf-dm_alerta-send_notification-20 → Límite de 20 ejecuciones concurrentes en "send_notification".
3 lambdaConcur-adm-backend-data_ingestion-100 → Límite de 100 ejecuciones concurrentes en "data_ingestion".
```

8. Configuración de Tiempo de Espera (Timeout)

El timeout define el tiempo máximo que una función puede ejecutarse.

Formato:

```
1 lambdaTimeout-<siglaVerticalNegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

```
1 lambdaTimeout-col-calimaco-process_order-15s → Tiempo de espera de 15s para "process_order".
2 lambdaTimeout-caf-dm_alerta-send_notification-10s → Timeout de 10s para "send_notification".
3 lambdaTimeout-adm-backend-data_ingestion-30s → Timeout de 30s para "data_ingestion".
```

9. Variables de Entorno (Environment Variables)

Las variables de entorno almacenan configuraciones dinámicas para Lambda.

Formato:

```
1 lambdaEnv-<siglaVerticalNegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

```
1 lambdaEnv-col-calimaco-db_url → Variable de entorno para la URL de base de datos en "calimaco".
2 lambdaEnv-caf-dm_alerta-s3_bucket_name → Variable con el nombre del bucket S3 en "dm_alerta".
3 lambdaEnv-adm-backend-auth_token → Variable con el token de autenticación para el backend.
```

3.4 Servicios API Gateway

Existen distintos tipos de API que pueden implementarse mediante Amazon API Gateway, cada uno con características específicas según el caso de uso. La nomenclatura debe reflejar el tipo de API que se implementa para mejorar la trazabilidad y facilitar su gestión en entornos mixtos.

Tipos de API:

Tipo de API	Prefijo sugerido	Descripción
HTTP API	httpapi	API ligera y de bajo costo, recomendada para casos simples y de alto rendimiento.

REST API pública	restapi	API REST completa, con funcionalidades avanzadas de gestión, transformación y seguridad.
REST API privada	restapi-pvt	API REST que solo puede ser accedida desde una VPC.
WebSocket API	wsapi	API bidireccional para casos en tiempo real como chats o dashboards.

1. API (HTTP, REST o WebSocket)

Formato:

```
1 <tipoapi>-<siglaverticalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-personalizado>
```

Ejemplos:

- httpapi-col-calimaco-files → HTTP API para manejo de archivos.
- restapi-col-calimaco-mantenimientos → API REST pública para operaciones de mantenimiento.
- restapi-pvt-col-calimaco-backend → API REST privada solo accesible desde una VPC.
- wsapi-col-calimaco-notificaciones → WebSocket API para notificaciones en tiempo real.

2. Recursos API (Resources)

Formato sugerido:

```
1 apires-<siglaverticalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-endpoint>
```

Ejemplos:

- apires-col-calimaco-mantenimientos-juegos
- apires-col-calimaco-reportes-diarios
- apires-col-calimaco-files-subir
- apires-col-calimaco-notificaciones-conexion

3. Métodos API (Methods)

Formato sugerido:

```
1 apimethod-<siglaverticalnegocio>-<nombre-proyecto | identificador-recurso>-<nombre-endpoint>-<verbo>
```

Ejemplos:

- apimethod-col-calimaco-mantenimientos-juegos-post
- apimethod-col-calimaco-reportes-diarios-get
- apimethod-col-calimaco-files-subir-put
- apimethod-col-calimaco-notificaciones-conexion-post

4. Deployment de API

Formato:

```
1 apideploy-<siglaverticalnegocio>-<nombre-proyecto | identificador-recurso>-<stage>
```

Ejemplos:

- apideploy-col-calimaco-mantenimientos-dev
- apideploy-col-calimaco-files-prod
- apideploy-col-calimaco-notificaciones-test

5. Stages

Formato:

```
1 apistage-<siglaverticalnegocio>-<nombre-proyecto | identificador-recurso>-<stage>
```

Ejemplos:

- `apistage-col-calimaco-mantenimientos-dev`
 - `apistage-col-calimaco-files-prod`
 - `apistage-col-calimaco-notificaciones-test`
-

6. Integraciones

Formato sugerido:

```
1 apiint-<siglaververticalnegocio>-<nombre-proyecto | identificador-recurso>-<tipo-backend>-<nombre-personalizado>
```

Ejemplos:

- `apiint-col-calimaco-mantenimientos-lambda-crear_juego`
- `apiint-col-calimaco-files-http-storage_service`
- `apiint-col-calimaco-notificaciones-lambda-envio_evento`